

Data Structures and Algorithms

How to reverse singly linear linked list?

```
void reverse() {
    Node oldhead = head;
    head = null;
    while(oldhead != null) {
        // delete first node from old list
        Node temp = oldhead;
        oldhead = oldhead.next;
        // add that node at the start of new list
        temp.next = head;
        head = temp;
    } // repeat until old list is empty
}
```

```
void reverse() {
    Node *oldhead = head;
    head = NULL;
    while(oldhead != NULL) {
        // delete first node from old list
        Node temp = oldhead;
        oldhead = oldhead->next;
        // add that node at the start of new list
        temp->next = head;
        head = temp;
    } // repeat until old list is empty
}
```

How to reverse singly linear linked list using recursion?

```
Node reverse(Node cur) {  
    // if cur node is last node of list, then head to that node  
    if(cur.next == null) {  
        head = cur;  
        return cur;  
    }  
    // reverse the rest of the list  
    Node tail = reverse(cur.next);  
    // add cur node to the next of tail  
    tail.next = cur;  
    // make cur next null  
    cur.next = null;  
    // cur node is last node for the prev call  
    return cur;  
}  
Node reverse() {  
    if(head != null)  
        reverse(head);  
}
```

How to reverse singly linear linked list using recursion -- with single pointers?

```
Node reverse(Node cur) {  
    // if cur node is last node of list, then head to that node  
    if(cur.next == null) {  
        head = cur;  
        return cur;  
    }  
    // reverse the rest of the list and add cur node to the next of tail  
    reverse(cur.next).next = cur;  
    // make cur next null
```

```
    cur.next = null;
    // cur node is last node for the prev call
    return cur;
}
Node reverse() {
    if(head != null)
        reverse(head);
}
```

```
Node reverse(Node cur) {
    if(cur.next == null)
        head = cur;
    else
        reverse(cur.next).next = cur;
    cur.next = null;
    return cur;
}
Node reverse() {
    if(head != null)
        reverse(head);
}
```

Display singly linear linked list in reverse order.

- Time: $O(n^2)$

```
int count = 0;
// count number of nodes
for(Node trav = head; trav != null; trav = trav.next)
    count++;
while(count > 0) {
    // traverse upto count position
```

```
Node trav = head;
for(int i=1; i<count; i++)
    trav = trav.next;
// display that node
System.out.println(trav.data);
// decrement count
count--;
} // repeat until count is 0
```

Display singly linear linked list in reverse order.

- Time: $O(n)$
- Aux Space: $O(n)$

```
// push all elements on stack
Stack<Integer> s = new Stack<>();
for(Node trav=head; trav!=null; trav=trav.next)
    s.push(trav.data);
// pop all elements from stack and print them -- one by one
while(!s.isEmpty()) {
    int ele = s.pop();
    System.out.println(ele);
}
```

Display singly linear linked list in reverse order -- using recursion.

```
void reverseDisplay(Node cur) {
    // if cur node is null, do nothing
    if(cur == null)
        return;
    // display rest of the list
```

```
reverseDisplay(cur.next);
// display the current node
System.out.println(cur.data);
}
void reverseDisplay() {
    reverseDisplay(head);
}
```

Display a given range of numbers in reverse order -- using recursion.

```
void revDisplay(int s, int e) {
    // if all nums are done, do nothing
    if(s > e)
        return;
    // display next numbers
    revDisplay(s+1, e);
    // display current number
    System.out.println(s);
}
```

Find the middle node of singly linear linked list.

- Time: $O(n)$ -- Needs $n/2$ iterations

```
Node findMid() {
    Node slow = head, fast = head;
    while(fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```

Find the middle node of singly linear linked list using single pointer.

- Time: $O(n)$ -- But needs $n+n/2$ iterations
- Traverse till last node and count the number of nodes.
- Traverse till $\text{count}/2$ and print it.

Find the middle node of singly linear linked list using recursion.

- Refer slides

Check if singly linear list contains a loop.

```
boolean hasLoop() {  
    Node slow = head, fast = head;  
    while(fast != null && fast.next != null) {  
        slow = slow.next;  
        fast = fast.next.next;  
        if(slow == fast)  
            return true;    // loop  
    }  
    return false;    // no loop  
}
```

Check if given singly linear list is palindrome or not.

- Time: $O(n)$
- Space: $O(n)$

```
boolean isPalindrome() {  
    Stack<Integer> s = new Stack<>();
```

```
for(Node trav=head; trav!=null; trav=trav.next)
    s.push(trav.data);
for(Node trav=head; trav!=null; trav=trav.next)
    if(trav.data != s.pop())
        return false; // not palindrome
return true;
}
```

Check if given singly linear list is palindrome or not.

- Find the middle of the list -- fast and slow pointer.
- Reverse the second half of the list.
- Compare first half of the list with second half of the list (reversed). If same, palindrome, otherwise not.

Create a stack (LIFO) using queue (FIFO).

- Stack operations: push(), pop(), peek(), isEmpty().

```
class Stack {
    private Queue<Integer> main = new LinkedList<>();
    private Queue<Integer> temp = new LinkedList<>();
    // time: O(n)
    public void push(int val) {
        while(!main.isEmpty()) {
            int ele = main.poll(); // pop
            temp.offer(ele); // push
        }
        main.offer(val); // push
        while(!temp.isEmpty()) {
            int ele = temp.poll(); // pop
            main.offer(ele); // push
        }
    }
    // time: O(1)
}
```

```
public int pop() {  
    return main.poll(); // pop  
}  
// time: O(1)  
public boolean isEmpty() {  
    return main.isEmpty();  
}  
}
```

Create a queue using stack.

- Homework

Input a string from user and find the character repeated maximum number of times (irrespective of case).

- Hint: Use Hashing.

Print the length of highest continous number range in given array.

- Input: 8, 2, 9, 1, -3, 5, 4, 3, 7, -2
- Output: 5 (because highest continous range: 1, 2, 3, 4, 5 -- 5 elements)
- Hint: Use Hashing (HashSet).

```
static int findLongestConseqSubseq(int arr[]) {  
    HashSet<Integer> set = new HashSet<>();  
    for(int ele: arr)  
        set.add(ele);  
    int maxseq = 0;  
    for(int ele: set) {  
        int cnt = 0;  
        while(set.contains(ele)) {  
            cnt++;  
            ele++; // check next element  
        }  
    }  
}
```

```
        if(cnt > maxseq)
            maxseq = cnt;
    }
    return maxseq;
}
```

Core Java

What is wrapper class? What is their use?

- Use primitive types as objects
 - used in collection
- Data-type conversion
 - non-static methods: intValue(), byteValue(), longValue(), floatValue(), doubleValue()
 - static methods: parseInt(), parseDouble(), valueOf(), ...
- Helper methods
 - Integer.max(), Integer.sum(), ...
- Information of primitive types
 - e.g. Integer.BYTES = 4 bytes (int size), Integer.MAX_VALUE (max int), ...
- Auto-boxing

Which are methods of java.lang.Object class? Which are native methods of object class?

- native protected Object clone()
 - Creates and returns a copy of this object.
- boolean equals(Object obj)
 - Indicates whether some other object is "equal to" this one.
- protected void finalize()
 - Called by the garbage collector on an object when garbage collection determines that there are no more references to the object.
- native Class<?> getClass()
 - Returns the runtime class of this Object.
- native int hashCode()
 - Returns a hash code value for the object.
- native void notify()
 - Wakes up a single thread that is waiting on this object's monitor.
- native void notifyAll()

- Wakes up all threads that are waiting on this object's monitor.
- String toString()
 - Returns a string representation of the object.
- void wait()
 - Causes the current thread to wait until another thread invokes the notify() method or the notifyAll() method for this object.
- void wait(long timeout)
 - Causes the current thread to wait until either another thread invokes the notify() method or the notifyAll() method for this object, or a specified amount of time has elapsed.
- void wait(long timeout, int nanos)
 - Causes the current thread to wait until another thread invokes the notify() method or the notifyAll() method for this object, or some other thread interrupts the current thread, or a certain amount of real time has elapsed.

What is the need of package? Which types are allowed to declare in package?

- Packages makes Java code modular. It does better organization of the code.
- Package is a container that is used to group logically related classes, interfaces, enums, and other packages.
- Package helps developer:
 - To group functionally related types together.
 - To avoid naming clashing/collision/conflict/ambiguity in source code.
 - To control the access to types.
 - To make easier to lookup classes in Java source code/docs.

Why we can not declare multiple public classes in single .java file?

- Rule: public class name must be same as file name.
- Efficient compilation process (Faster linking during import).

Why Strings in Java are immutable?

- Since strings are immutable, same string literals can be given same memory in JVM.
 - This optimize memory required for execution of Java program.
 - String s1 = "SUNBEAM";
 - String s2 = "SUNBEAM";

- Similar string literals are internally stored in String pool.
- During execution, JVM maintains all class names, method names, and other identifiers as Strings.
 - If Strings can be modified, there is possibility that these names of identifiers can be changed on the fly (dynamically). This may lead to unexpected execution.
 - Immutable strings avoid any such threats/attacks.

What is the difference between import and static import?

- import is used to import public types from other packages.

```
import java.util.ArrayList;
import java.io.*;
```

- static import is used to import accessible static members of any class.

```
import static java.lang.Math.*;
import static java.lang.System.out;

class Test {
    public static void main(String[] args) {
        double result = sqrt(7.0);
        out.println(result);
    }
}
```

How can we pass argument to the method by reference? Explain with example?

- By value: Copy of argument is passed to function
 - Any changes in variable inside called function will not reflect in calling function.
 - Passing primitive types are always by value in Java.
- By reference: Reference (address) of argument is passed to function

- Any changes in variable inside called function will reflect in calling function.
- Passing reference types (class objects, arrays) are always by reference in Java.

```
class MyInt {  
    private int a;  
    // getter/setter  
}  
  
class Tester {  
    // by ref  
    public static void fun(MyInt x) {  
        int i = x.get() * 2;  
        x.set(i);  
    }  
    // by value  
    public static void fun(int x) {  
        x = x * 2;  
    }  
}
```

What is the difference between checked and unchecked exception?

- Java compiler expect that certain exception must be handled by the programmer -- checked exception.
 - catch block
 - throws clause
- Most of the checked exceptions arise out of the JVM. Hence Java wants programmer to be aware of them and handle them. For example:
 - File IO --> IOException
 - Database --> SQLException
 - Threads --> InterruptedException
- The other exceptions usually arise due to programmers/users mistakes and within the JVM. For example:
 - NullPointerException
 - ArithmeticException

Which are the advantages and disadvantages of generics?

- Advantages:
 - Type-safety
- Disadvantages/Limitations:
 - Doesn't work with primitive types
 - Can't create object/array of generic type
 - Can't overload based on generic type difference
 - Static fields cannot be of generic type param.
 - Can't cast or instanceof with generic type.
 - Can't throw or catch objects of generic type.
- Type Erasure: The generic type param info is not maintained at runtime (inside JVM).

```
ArrayList<Integer> list1 = new ArrayList<>();  
// Internally (inside JVM), it creates ArrayList of "Object"s.  
// And compiler ensures that only Integer can be added to it -- Typesafety.  
ArrayList<String> list2 = new ArrayList<>();
```

```
void method(ArrayList<Integer> list) {  
    // ...  
}  
// error: such overloading is not allowed (with different type param only)  
void method(ArrayList<String> list) {  
    // ...  
}
```

What is difference between Comparable and Comparator?

- Comparable
 - The current object (**this**) is Comparable to the other object -- meant to be written in the class to be compared.

- java.lang.Comparable
- Natural ordering

```
class Student implements Comparable<Student> {  
    // ...  
    public int compareTo(Student other) {  
        int diff = this.roll - other.roll;  
        return diff;  
    }  
}
```

```
Student[] arr = { .... };  
Arrays.sort(arr); // Comparable.compareTo()
```

- Comparator
 - An object compares two other objects.
 - java.util.Comparator
 - Custom order

```
class StudentComparator implements Comparator<Student> {  
    public int compare(Student s1, Student s2) {  
        int diff = s1.marks - s2.marks;  
        return diff;  
    }  
}
```

```
Student[] arr = { .... };  
Arrays.sort(arr, new StudentComparator()); // StudentComparator.compare()
```

```
//Arrays.sort(arr, (s1, s2) -> s1.marks - s2.marks);
```

- Multi-level sorting/comparison

```
class Emp {  
    // ...  
}  
  
class Main {  
    public static void main(String... args) {  
        // arr  
        Arrays.sort(arr, new Comparator<>() {  
            @Override  
            public int compare(Emp e1, Emp e2) {  
                int diff = e1.getDept() - e2.getDept();  
                if(diff == 0)  
                    diff = e1.getJob().compareTo(e2.getJob());  
                if(diff == 0)  
                    diff = Double.compare(e1.getSal(), e2.getSal());  
                return diff;  
            }  
        });  
        // ...  
    }  
}
```

What is difference between ArrayList and Vector? How to use ListIterator?

- V: Legacy collection 1.0
- A: Collection framework 1.2
- V: Synchronized -- Thread safe -- Slower -- Suitable in multi-thread applns

- A: Non-Synchronized -- Non thread safe -- Faster -- Suitable in single-thread applns
- V: Enumeration (later added support of Iterator)
- A: Iterator
- V: Initial size 10 & capacity grow (double)
- A: Initial size 10 & capacity grow (+half)
- V: Dynamically growing array
- A: Dynamically growing array
- ListIterator -- Bidirectional traversal

```
ListIterator<String> itr = list.listIterator();  
while(itr.hasNext()) {  
    String ele = itr.next();  
    // ...  
}
```

```
ListIterator<String> itr = list.listIterator(list.size());  
while(itr.hasPrevious()) {  
    String ele = itr.previous();  
    // ...  
}
```

What is fail-fast and fail-safe iterator?

- Fail Fast ... While processing iterator if any element is modified, ConcurrentModificationException is thrown.

```
java.util.ArrayList<String> list = new ArrayList<>();
// ...

// thread 1
Iterator<String> itr = list.iterator();
while(itr.hasNext()) {
    String ele = itr.next();
    // ...
}

// thread 2
list.set(4, newvalue);
```

- Fail Safe ... While processing iterator if any element is modified, modification exception is not raised.

```
List<String> list = new java.util.concurrent.CopyOnWriteArrayList<>();
// ...

// thread 1
Iterator<String> itr = list.iterator();
while(itr.hasNext()) {
    String ele = itr.next();
    // ...
}

// thread 2
list.set(4, newvalue);
```

How to make ArrayList Synchronized?

```
List<String> list = Collections.synchronizedList(new ArrayList<String>());  
// internally create a proxy/wrapper that includes synchronized methods which in turn call ArrayList methods.
```

What is serialization and deserialization? What is significance of serialVersionUID?

- **Serialization:** Converting state of object into sequence of bytes. It includes state (field values) as well as class info. This state can be stored in file or sent over network. For this, class must be Serializable.

```
objectOutputStream.writeObject(obj);
```

- **Deserialization:** Constructing object back from the state stored as sequence of bytes.

```
obj = objectInputStream.readObject();
```

- **serialVersionUID**

```
class Transaction {  
    static final long serialVersionUID = 2L;  
    // fields ...  
    // methods  
}
```

- The serialization runtime associates with each serializable class a version number, called a serialVersionUID, which is used during deserialization to verify that the sender and receiver of a serialized object have loaded classes for that object that are compatible with respect to serialization. If the receiver has loaded a class for the object that has a different serialVersionUID than that of the corresponding sender's class, then deserialization will result in an `InvalidClassException`.
- If a serializable class does not explicitly declare a serialVersionUID, then the serialization runtime will calculate a default serialVersionUID value for that class based on various aspects of the class, as described in JLS. However, it is strongly recommended that all serializable classes explicitly declare serialVersionUID values.

Deep copy vs Shallow copy

- Shallow copy -- Created by Object.clone() method for Cloneable objects.
 - Only copies current object (not the embedded objects i.e. objects referenced by fields in the object).

```
class Human implements Cloneable {  
    int age;  
    String name;  
    Date birth;  
    // ...  
    @Override // shallow copy  
    Object clone() throws ... {  
        Human other = super.clone(); // Object.clone()  
        return other;  
    }  
}
```

- Copies the whole object including embedded objects. Needs to be done explicitly in overridden clone() method.

```
class Human implements Cloneable {  
    int age;  
    String name;  
    Date birth;  
    // ...  
    @Override // deep copy  
    Object clone() throws ... {  
        Human other = super.clone(); // Object.clone()  
        other.birth = this.birth.clone();  
        return other;  
    }  
}
```

What is functional interface? Which functional interfaces are predefined and where they are used?

- Functional interface: SAM (Single Abstract Method)
 - Any number of default methods
 - Any number of static methods
- @FunctionalInterface -- compiler check for SAM - if 0 or multiple SAM, then compiler error.
- (Before Java 8) Functional Interface:
 - Comparable, Comparator, Runnable, Closeable, ...
- (In Java 8) Functional Interfaces -- java.util.function.*
 - Predicate ... boolean test(T val);
 - used in filter() operation
 - Function ... R apply(T val);
 - used in map() operation
 - Consumer ... void accept(T val);
 - used in forEach() operation
 - Supplier ... T get();
 - used in generator
- In lambda expressions, arguments are same as args of SAM of functional interface and return value is return value of that SAM.
- Functional Interfaces are also used for method references.
 - `Consumer<String> cons = System.out::println;`
 - `cons.accept("Sunbeam"); // --> System.out.println("Sunbeam");`

What is significance of filter(), map(), flatMap() and reduce() operations? In which scenarios they are used?

- java.util.Stream -- represents stream of data elements
 - No storage: Stream doesn't hold data like collections. It processes data from collections/arrays.
 - Immutable: `stream1 --> Intermediate operation --> stream2`
 - Lazily evaluated: `stream1.iop1().iop2().iop3().iop4().iop5().terminal();`
 - Not reusable: Can perform only one terminal operation on stream.
- Stream operations
 - Intermediate operations e.g. map(), filter(), sort(), ...
 - Terminal operations e.g. forEach(), collect(), reduce(), ...

- Examples:

```
Integer[] array = {1, 2, 3, 4, 5, 6, 7, 8, 9};
Stream<Integer> stream = Stream.of(array);
stream                                     // 1, 2, 3, 4, 5, 6, 7, 8, 9
    .filter(i -> i % 2 != 0)              // 1, 3, 5, 7, 9
    .map(i -> "DAC"+i)                    // DAC1, DAC3, DAC5, DAC7, DAC9
    .forEach(s -> System.out.println(s));
```

```
Integer[] array = {1, 2, 3, 4, 5, 6, 7, 8, 9};
Stream<Integer> stream = Stream.of(array);
List<String> list = stream
    .filter(i -> i % 2 != 0)              // 1, 3, 5, 7, 9
    .map(i -> "DMC"+i)                   // DMC1, DMC3, DMC5, DMC7, DMC9
    .collect(Collectors.toList());
for(String str : list)
    System.out.println(str);
```

- <https://winterbe.com/posts/2014/03/16/java-8-tutorial/>
- <https://winterbe.com/posts/2014/07/31/java8-stream-tutorial-examples/>

Functional Programming advantages

- Concise code
- Pure functions -- Output solely depends on input (isolated -- side-effect free)

- Lazy evaluated -- better performance
- More readable
- Suitable for parallel/distributed programming

What is use of reflection? Why reflection is slower?

- Reflection uses
 - Get metadata of the types (associated in a java.lang.Class object)
 - getName(), getSuperclass(), getMethods(), getFields(), getConstructors(), getAnnotations()
 - Invoke methods dynamically or access fields dynamically

```
Object obj = cls.newInstance();
Method m = cls.getDeclaredMethod("methodName", argTypes);
Object result = m.invoke(obj, args);
```

- Invoke method without reflection

```
ClassName obj = new ClassName();
Object result = obj.methodName(args);
```

- Reflection is slower, because it needs to find the method at runtime and then invoke it on dynamically created object.

difference between Closeable and finalize().

- When GC collects unused object, it will invoke finalize() method.
- The finalize() should be overridden in your class to cleanup the resources.

```
class MyDao {
    Connection con;
    // ...
}
```

```
public void finalize() {  
    con.close();  
}  
}
```

- However, you cannot force/guarantee the GC. Possibly if GC is delayed, your resource remains open for longer duration. Poor performance.
- Closeable implementation ensure that resource can be closed immediatly after its use.

```
class MyDao implements Closeable {  
    Connection con;  
    // ...  
    public void close() {  
        con.close();  
    }  
}
```

```
MyDao dao = new MyDao();  
// ...  
dao.close(); // resource closed immediatly
```

What is relation between Thread `start()` and `run()` method?

- `th.start()` --> submit the JVM thread to the (jvm) scheduler.
- When scheduler schedules your thread, it (jvm) invokes `run()` method of the thread.

When we should create thread by implementing Runnable and extending Thread class?

- Java support multiple inheritance of interfaces, but not of classes. If class is already inherited from another class, it can't be inherited from Thread class. There one have to use Runnable interface.

```
class MyThread extends Thread {  
    @Override  
    public void run() {  
        // ...  
    }  
    public void startWork() {  
        Thread t = new MyThread();  
        t.start();  
    }  
}
```

```
class MyWindow extends JFrame implements Runnable {  
    @Override  
    public void run() {  
        // ...  
    }  
    public void startWork() {  
        Thread t = new Thread(this); // this object is inherited from Runnable  
        t.start();  
    }  
}
```

wait() and notify()

```
```Java  
// thread1
synchronized(obj) {
 // ... task A
 obj.notify();
 // 1. wake-up one of the thread sleeping on the object lock.
```

```
}
...

```Java  
// thread2  
synchronized(obj) {  
    obj.wait();  
    // ... task B (to be done after A)  
    // 1. release the lock on obj.  
    // 2. block the current thread on the lock until it is notified.  
    // 3. after wake-up lock obj again and resume execution.  
}  
...`
```

Object Oriented Programming

What are object oriented concepts? What is difference between object-based, object-oriented and fully object-oriented language?

- Pillars of Object Oriented Programming ... Grady Booch (Object Oriented Analysis Design)
 - Major Pillars ... Must be supported by any Object Oriented language
 - Abstraction
 - Encapsulation
 - Hierarchy ... Reusability
 - is-a (inheritance)
 - Person <|-- Employee <|-- Manager
 - has-a (association)
 - Computer <-- Motherboard <-- Processor
 - use-a (dependency)
 - MyLinkedList "use a" Stack class for reversing list.
 - creates-a (instantiation)
 - SessionFactory creates Session.
 - Modularity
 - Divide source code into multiple logical units ... better maintainability
 - Division
 - Source code files
 - Classes
 - Packages, Namespaces, Modules
 - Minor Pillars ... May be supported by any Object Oriented language
 - Typing
 - Strong vs Weak -- Data types
 - Java: Strong typing
 - JS: Weak typing
 - Static vs Dynamic -- Polymorphism
 - Compile time

- Runtime
 - Concurrency
 - Object behaviour in case of multi threading (how it work with race condition?)
 - Persistence
 - Maintaining state of object outside its scope (e.g. database, file - serialization, network)
- Object Based
 - Supports only abstraction & encapsulation (i.e. classes/objects)
 - e.g. Visual Basic, Old Java script, ...
- Object Oriented
 - Must support all major Pillars
 - e.g. C++, Java
- Fully Object Oriented
 - Must support all major and minor Pillars
 - Each data must be object (even primitive types must be objects)
 - No global variables/functions
 - e.g. C#

What is abstraction and encapsulation. Give real-life example.

- Abstraction
 - Getting essential details of the system.
 - Hides unnecessary (from user perspective) details.
 - Depends on perspective of user.
 - Outer view of the system.
 - Implemented with abstract classes and interfaces.
- Encapsulation
 - Binding code and data together in a class.
 - Inner view of the system.
 - Hiding data to achieve security.
 - Abstraction & Encapsulation are complementary to each other.
 - Implemented with private, protected members.
- Implementation

```
interface Account {
    public void deposit(double amount);
}
class BankAccount implements Account {
    private double balance;
    public double getBalance() {
        return this.balance;
    }
    public void deposit(double amount) {
        this.amount += balance;
    }
}
```

Feature	Abstraction	Encapsulation
Definition	Hiding unnecessary details and exposing only relevant features.	Hiding data and restricting direct access by binding it with methods.
Focus	Focuses on design – what should be exposed.	Focuses on implementation – how data should be protected.
Purpose	Simplifies complexity by providing a clear interface.	Protects data by preventing unintended modifications.
How is it achieved?	Using abstract classes, interfaces (OOP) or modular design (functional programming).	Using private/protected members and public methods (getters/setters) .
Perspective	Outer view (What the user sees).	Inner view (How data is stored and manipulated).
Example	A car driving system hides complex mechanics but provides a simple interface (steering, brake, accelerator).	The car engine encapsulates internal components to ensure safe operation.

What are advantages of Object Oriented Programming?

- Data security

```
struct Time {  
    int hour, minute, seconds;  
};  
// ....  
struct Time t1;  
t1.seconds = 100; // no compiler error -- may cause business logic to fail  
// ...  
incTime(&t1);
```

```
class Time {  
    private int hour, minute, seconds;  
    // methods ... setters/getters  
    public void setSeconds(int sec) {  
        if(sec < 0 || sec >= 60)  
            throw exception;  
        seconds = sec;  
    }  
};  
// ....  
struct Time t1;  
t1.setSeconds(100); // throw exception  
// ...  
t1.incTime();
```

- Modularity
 - Namespaces/Packages/Modules
 - Classes
- Reusability (Association & Inheritance)
 - Classes are reused

What is class and object? Give real-life example.

- Class is blue-print of the object. Class has fields (data) and methods (operations).
- Object is an instance of the class. One class can have multiple objects.
- Class is a logical entity.
- Object is a "physical" entity. Allocates memory.

What are characteristics of object? Explain them.

- State -- Values of data members
- Behaviour -- Operations performed/Methods called on state
- Identity -- Uniqueness of the object (e.g. address)

What is the need of getter and setter functions in class?

- Controlled access to the private fields
 - Programmer can choose which fields need to provide getter, setter or both.
 - Programmer can add validation logic into setter methods.

What is polymorphism? What are its types? Explain them with examples. What is the difference between function overloading and function overriding?

- Polymorphism -- Taking many forms.
 - Same name -- Different implementations
- Compile time/Static Polymorphism
 - Function with same name but different arguments in same scope -- Function overloading.
 - Internally compiler assigns different name to each method (Name mangling).
 - Compiler decides which function to be called (Early binding), depending on number/type of arguments.

```
int add(int a, int b) { // addii
    return a + b;
}
int add(int a, int b, int c) { // addiii
```

```
    return a + b + c;
}
```

```
res = add(11, 22);
res = add(11, 22, 33);
```

- False polymorphism
- Run time/Dynamic Polymorphism
 - Function with same name and same arguments redefined in derived class -- Function overriding.
 - Internally uses method table/virtual function table -- dynamic method dispatch.
 - Function to be called is decided at runtime (Late binding), depending on type of object (not the reference).

```
class Base {
    public void print() {
        System.out.println("Base.print() called");
    }
}
class Derived extends Base {
    public void print() {
        //super.print(); // Base::print();
        System.out.println("Derived.print() called");
    }
}
```

```
Base obj1 = new Base();
obj1.print(); // call Base.print()
Base obj2 = new Derived();
obj2.print(); // call Derived.print()
```

- True polymorphism

What is function overloading? Which are the rules of function overloading? Why return type is not considered in function overloading?

- Function with same name but "different" arguments in same scope -- Function overloading.
- Arguments should differ in count, data type, and/or order.
- Return type is not considered, because collecting return value is not compulsory. Compiler will not be able to resolve the function to be called.

```
int divide(int a, int b) {  
    return a / b;  
}  
  
double divide(int a, int b) {  
    return (double)a / b;  
}
```

```
int r1 = divide(15, 5);  
double r2 = divide(15, 5);
```

```
divide(15, 5);
```

What are different types of hierarchy? When to use which one?

- is-a hierarchy -- Inheritance (kind of)
 - Inheritance (Generalization -> Specialization)
 - Student is a person.
 - ~~Student has a person.~~
- has-a hierarchy -- Association (part of)
 - Represents a whole-part relationship where one object contains another.

- Achieved via association, which can be:
 - Composition (Strong ownership, lifecycle dependency)
 - Aggregation (Weaker relationship, independent lifecycle)
- Employee has a Joining date.
- ~~Employee is a Joining date.~~
- use-a hierarchy -- Dependency
 - Represents a temporary relationship where an object depends on another to perform an action.
 - Achieved via method parameters, function calls, or local objects.
 - When an external service or utility class is used.
 - Document uses Printer (to print).
- creates-a hierarchy -- Instantiation/Factory
 - Represents an object-creation relationship, where one class instantiates another.
 - ProductFactory creates Product.

Why constructor is considered special member function of the class?

- Constructor has same name as of class name.
- Constructor doesn't have any explicit return type.
- Constructor is automatically called when object is created.

What is object slicing? Explain object slicing in context of upcasting?

- Assigning derived class object address to base class reference is called as "Up-casting".
- When derived class object is assigned to base class reference, through that base class reference one can access only the base class members. This is called "Object slicing".

```
Person p = new Employee();  
System.out.println(p.getName());  
System.out.println(p.getSalary()); // not allowed -- compiler error
```

What is down-casting and when it is required?

- Assigning base class reference to derived class reference is called as "Down-casting".

```
void displayStudent(Person p) {  
    System.out.println(p.getName());  
    Student s = (Student)p; // Down-casting  
    System.out.println(s.getMarks()); // allowed  
}
```

```
displayStudent(new Student(...));
```

```
void displayEmployee(Person p) {  
    System.out.println(p.getName());  
    Employee e = (Employee)p; // Down-casting  
    System.out.println(e.getSalary()); // allowed  
}
```

```
displayEmployee(new Employee(...));
```

```
void display(Person p) {  
    System.out.println(p.getName());  
    if(p instanceof Employee) {  
        Employee e = (Employee)p; // Down-casting  
        System.out.println(e.getSalary()); // allowed  
    }  
    if(p instanceof Student) {  
        Student s = (Student)p; // Down-casting  
    }  
}
```

```
        System.out.println(s.getMarks()); // allowed
    }
}
```

```
display(new Person(...));
display(new Student(...));
display(new Employee(...));
```

What do you know about association, composition and aggregation. Explain with the help of example.

- has-a relationship -- Association
- Composition (Tight coupling) -- part of

```
class engine {
    // ...
};
class car {
    engine e; // tight coupling
    // ...
};
```

- Aggregation (Loose coupling)

```
class driver {
    // ...
};
class car {
    driver *d; // loose coupling
```

```
// ...  
};
```

What are different types of inheritance? Explain with the help of example. What are problems with multiple inheritance?

- Inheritance represents: is-a relation.
- Generalization to Specialization.
- Person --> Employee --> Manager --> HRManager
- Person --> Student
- Types
 - Single inheritance
 - Multiple inheritance
 - Multi-level inheritance
 - Hierarchical inheritance
 - Hybrid inheritance
- Problems in multiple inheritance: Ambiguity

```
class bird {  
public:  
    void fly() {  
        // ...  
    }  
};  
class aeroplane {  
public:  
    void fly() {  
        // ...  
    }  
};  
class birdplane: public bird, public aeroplane {  
};
```

```
birdplane b;  
b.fly(); // Ambiguity: bird::fly() or aeroplane::fly() --> Compiler Error  
// b.bird::fly(); --> bird::fly()  
// b.aeroplane::fly(); --> aeroplane::fly()
```

Which are the different types of design pattern? Explain singleton design pattern.

- Design pattern is standard solutions to well-known problems in software development/coding.
- Design patterns are categorized into **three main types**, as defined in the **Gang of Four (GoF)** book "*Design Patterns: Elements of Reusable Object-Oriented Software*".
 1. **Creational Patterns** → Object creation mechanisms.
 2. **Structural Patterns** → Composition of classes and objects.
 3. **Behavioral Patterns** → Communication between objects.

1. Creational Design Patterns (Managing Object Creation)

These patterns deal with **object instantiation** to provide flexibility and reusability.

Pattern	Purpose
Singleton	Ensures only one instance of a class exists.
Factory Method	Creates objects without specifying exact class.
Abstract Factory	Provides an interface for creating related objects.
Builder	Constructs complex objects step by step.
Prototype	Creates new objects by copying an existing object.

Example: Singleton Pattern (Ensures a single instance of a class)

```
class Singleton {
    private static Singleton instance; // Static instance
    private Singleton() {} // Private constructor

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

2. Structural Design Patterns (Organizing Object Relationships)

These patterns **deal with class and object composition** to ensure flexible structures.

Pattern	Purpose
Adapter	Converts interface of a class into another expected interface.
Bridge	Separates abstraction from implementation.
Composite	Composes objects into tree structures.
Decorator	Adds responsibilities dynamically to objects.
Facade	Provides a simplified interface to a complex system.
Flyweight	Shares object instances to save memory.
Proxy	Acts as a placeholder for another object.

Example: Adapter Pattern (Converting one interface to another)

```
// Existing interface
interface OldCharger {
    void chargeWithMicroUSB();
}

// New interface
interface NewCharger {
    void chargeWithUSB_C();
}

// Adapter class to make old charger compatible with new interface
class ChargerAdapter implements NewCharger {
    OldCharger oldCharger;

    public ChargerAdapter(OldCharger oldCharger) {
        this.oldCharger = oldCharger;
    }

    @Override
    public void chargeWithUSB_C() {
        oldCharger.chargeWithMicroUSB(); // Adapting old method
    }
}
```

3. Behavioral Design Patterns (Managing Object Interactions)

These patterns **focus on communication and behavior** among objects.

Pattern	Purpose
Observer	Notifies multiple objects when a state changes.
Strategy	Defines a family of algorithms, encapsulates them, and makes them interchangeable.

Pattern	Purpose
Command	Encapsulates a request as an object.
Mediator	Centralizes complex communication between objects.
Memento	Captures an object's state to restore later.
State	Changes behavior based on internal state.
Template Method	Defines a skeleton algorithm structure in a base class.
Iterator	Provides a way to access elements of a collection sequentially.
Visitor	Adds new operations to existing objects without modifying them.

Example: Observer Pattern (Notifies multiple objects of state changes)

- When you need a **one-to-many dependency** (e.g., event listeners, messaging systems).

```
import java.util.ArrayList;
import java.util.List;

// Observer interface
interface Observer {
    void update(String message);
}

// Concrete Observer
class User implements Observer {
    private String name;

    public User(String name) { this.name = name; }

    @Override
    public void update(String message) {
```

```
        System.out.println(name + " received notification: " + message);
    }
}

// Subject class
class Channel {
    private List<Observer> subscribers = new ArrayList<>();

    public void subscribe(Observer observer) { subscribers.add(observer); }
    public void unsubscribe(Observer observer) { subscribers.remove(observer); }

    public void notifyObservers(String message) {
        for (Observer observer : subscribers) {
            observer.update(message);
        }
    }
}

// Usage
public class ObserverPatternDemo {
    public static void main(String[] args) {
        Channel techChannel = new Channel();

        User user1 = new User("Nilesh");
        User user2 = new User("Prashant");

        techChannel.subscribe(user1);
        techChannel.subscribe(user2);

        techChannel.notifyObservers("New video uploaded!"); // Notifies all subscribers
    }
}
```

Common Design Patterns in Java Built-in Classes

Design Pattern	Java Built-in Class Example	Explanation
Singleton	<code>Runtime.getRuntime()</code>	Ensures a single instance of <code>Runtime</code> .
Factory Method	<code>Calendar.getInstance()</code>	Returns a subclass based on locale/time zone.
Builder	<code>StringBuilder</code> / <code>StringBuffer</code>	Allows step-by-step construction of mutable strings.
	<code>Stream.Builder</code>	Constructs Java streams flexibly.
Prototype	<code>Object.clone()</code>	Creates object copies using cloning.
Adapter	<code>java.util.Arrays.asList()</code>	Converts array to <code>List</code> (adapts different data structures).
	<code>InputStreamReader</code>	Converts <code>InputStream</code> to <code>Reader</code> .
	<code>OutputStreamWriter</code>	Converts <code>OutputStream</code> to <code>Writer</code> .
Decorator	<code>java.io.BufferedReader(new FileReader("file.txt"))</code>	Enhances <code>Reader</code> with buffering capabilities.
	<code>Collections.synchronizedList(list)</code>	Wraps list with synchronization.
Observer	<code>java.util.Observer</code> & <code>Observable</code> (Replaced by Event Listener)	Implements observer pattern for notifications.
	<code>PropertyChangeListener</code>	Used in JavaBeans for event handling.
Strategy	<code>Comparator</code> (with <code>Collections.sort()</code>)	Allows different sorting strategies.
	<code>ThreadPoolExecutor</code>	Supports different execution strategies via policies.
Command	<code>Runnable</code> & <code>Callable</code> (Used in <code>ExecutorService</code>)	Encapsulates actions as objects.
Proxy	<code>java.lang.reflect.Proxy</code>	Creates dynamic proxies for interfaces.



SUNBEAM

Sunbeam Infotech

Exploring new ideas, Reaching new heights!



Reverse display singly list

```
for (trav = head; trav != null; trav = trav.next)
    S.push(trav.data);
while (!S.isEmpty())
    print(S.pop());
```

$$\text{itrs} = n + n = 2n$$

$$T \propto 2n$$

$$T \propto n \rightarrow O(n)$$

$S \rightarrow$ aux space

$$S \propto n \rightarrow O(n)$$

cnt = 0;

```
for (trav = head; trav != null; trav = trav.next)
```

cnt++;

```
while (cnt >= 1) {
```

```
    trav = head;
```

```
    for (i = 1; i <= cnt; i++) {
```

```
        print(trav.data);
```

```
        trav = trav.next;
```

```
    }
```

```
    cnt--;
```

```
}
```

$S \rightarrow$ aux space

$$S = k \rightarrow O(1)$$

$$\text{itrs} = n + (n-1) + (n-2) + \dots + 1$$

$$\text{itrs} = \frac{n(n+1)}{2}$$

$$T \propto \frac{n^2 + n}{2}$$

$$T \propto n^2 + n$$

$$T \propto n^2$$

$$\underline{O(n^2)}$$

Recursion

- ① Explain process in terms of 'itself'.
- ② base/end condition.

factorial

$$n! = n * (n-1)! \rightarrow 0! = 1$$

power

$$x^y = x * x^{y-1} \rightarrow x^0 = 1$$

fibonacci

$$T_n = T_{n-1} + T_{n-2} \rightarrow \begin{matrix} T_1 = 1 \\ T_2 = 1 \end{matrix}$$

factors

factors(n) {

① if (n == 1)
return;

① find first prime
factor of n.

② print prime factor.

③ factors(n/prime)

print nums 1 to n in rev order

1 (2 3 4 5)

2 (3 4 5)

3 (4 5)

4 (5)

5

$i > n$

print(i) {
if (i > 5)
return;
print(i+1);
sop(i);

}

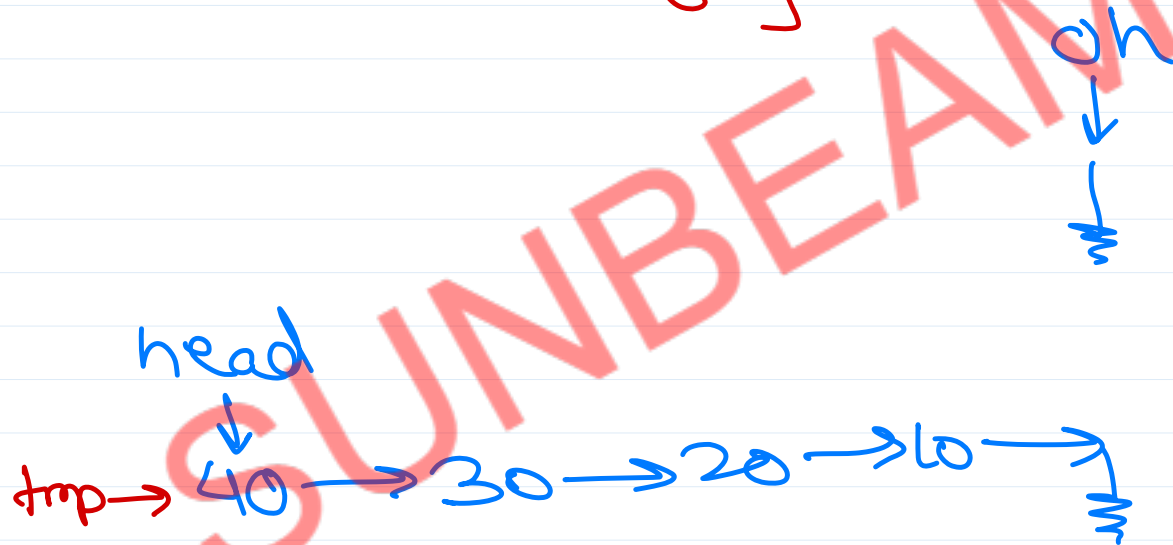
print(1);

reverse display singly list using recursion.

```
printRev(tvar) {  
    if (tvar == null)  
        return;  
    printRev(tvar.next);  
    print(tvar.data);  
}
```

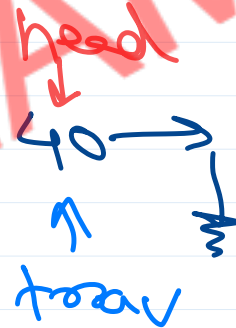
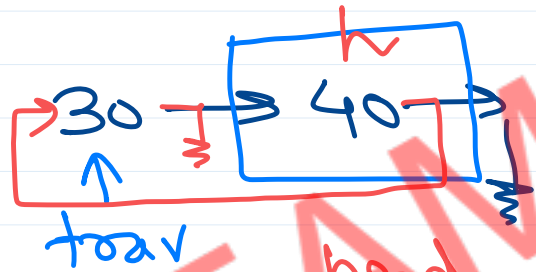
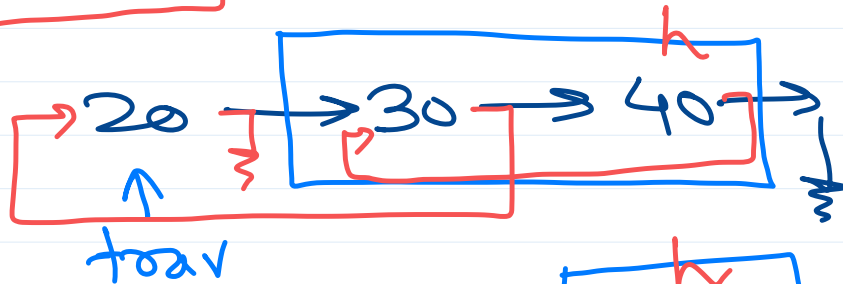
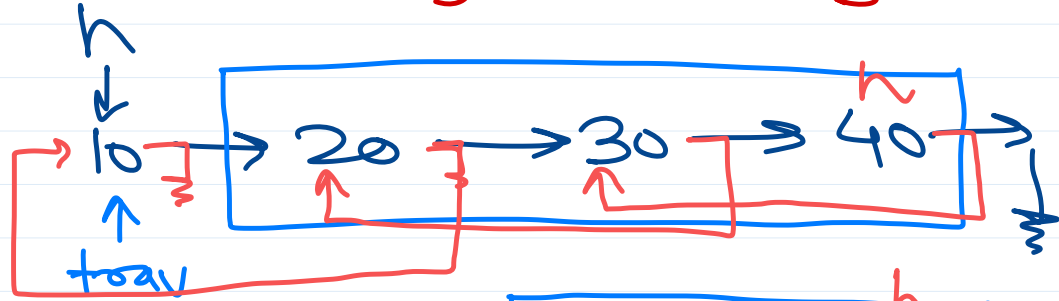
```
printRev(head);
```

reverse the singly linked list:



```
oh = h;  
h = null;  
while (oh != null) {  
    tmp = oh;  
    oh = oh.next;  
    tmp.next = h;  
    h = tmp;  
}
```

reverse singly list using recursion.



```

rev(toav) {
    if (toav.next == null)
        head = toav;
    else {
        rev(toav.next).next = toav;
        toav.next = null;
    }
}

```

- ③ if single node, make it head.
- ① reverse rest of list
- ② add toav (cur) to next of last node (of rev part)
- ③ toav next = null.

```

rev(toav) {
    if (toav.next == null)
        head = toav;
    else {

```

else {

last = rev(toav.next);

last.next = toav;

toav.next = null;

}

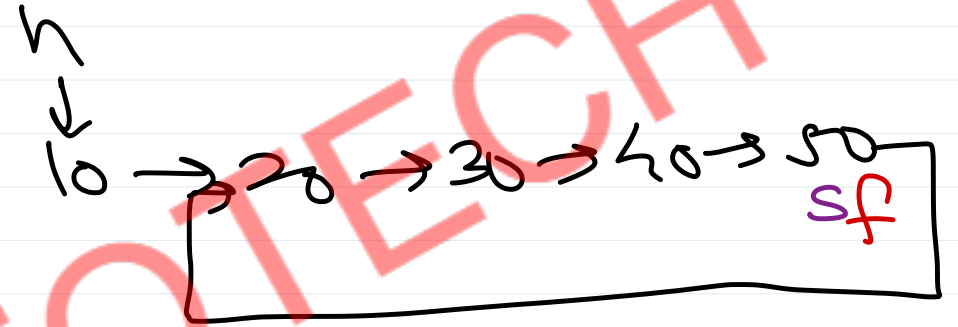
find mid of singly list.

```
pointMid (tvar, pos) {  
    if (tvar == null) {  
        max = pos;  
        return n;  
    }  
}
```

```
pointMid (tvar.next, pos + 1);  
if (pos == max / 2)  
    print (tvar.data);  
}
```

```
pointMid (head, 1);
```

check if list has loop.



Stack using queue

Soln: two queues

main
temp

Push(val) {

while (!m.isEmpty()) {

ele = m.poll();

t.offer(ele);

m.offer(val);

while (!t.isEmpty()) {

ele = t.poll();

m.offer(ele);

}
POP() {

return m.poll();
}

main:
10 20 30 40 →

temp:
→

Homework:

$T \propto 2n$
 $O(n)$

① Stack using Queue
push → $O(1)$
pop → $O(n)$

② Queue using Stack.

$O(1)$

count char repeated max num of times in given string.

```
map = new HashMap<char, int>();
```

```
for (i = 0; i < s.length; i++) {
```

```
    ch = s.charAt(i);
```

```
    cnt = map.getOrDefault(ch, 0);
```

```
    map.put(ch, cnt + 1);
```

```
}
```

```
maxch = s.charAt(0);
```

```
maxcnt = 0;
```

```
for (Entry en : map.entrySet()) {
```

```
    if (en.getValue() > maxcnt) {
```

```
        maxcnt = en.getValue();
```

```
        maxch = en.getKey();
```

```
    }
```

```
}
```

```
print(maxch);
```

longest max sub sequence. [2, 9, 7, 1, 8, 12, 4, 3]

```
set = new HashSet<>()
```

```
for (n: arr)
```

```
    set.add(n);
```

```
max = 0;
```

```
for (i = 0; i < arr.length; i++) {
```

```
    num = arr[i];
```

```
    len = 1;
```

```
    while (set.contains(++num))
```

```
        len++;
```

```
    if (len > max)
```

```
        max = len;
```

```
}
```

```
print(len);
```

1, 2, 3, 4 → 4

7, 8, 9 → 3

12 → 1

Sliding window problem

cars per hour = [5, 2, 8, 6, 10, 7, 1]
0 1 2 3 4 5 6

3-hrs window

$$w_1 \rightarrow 5 + 2 + 8 = 15$$

$$w_2 \rightarrow 2 + 8 + 6 = 16$$

$$w_3 \rightarrow 8 + 6 + 10 = 24$$

$$w_4 \rightarrow 6 + 10 + 7 = 23$$

$$w_5 \rightarrow 10 + 7 + 1 = 18$$

sum = 0;

for (i = 0; i < 3; i++)

sum += arr[i];

maxSum = sum;

for (i = 3; i < arr.len; i++) {

sum = sum + arr[i]
- arr[i - 3];

if (sum > maxSum)

maxSum = sum;

}

print(maxSum);



SUNBEAM

Thank you!

Nilesh Ghule <nilesh@sunbeaminfo.com>



1. What is the difference between malloc() vs new and free() vs delete?

- **malloc() vs. new:**

- **Type:**

- **malloc():** malloc() is a function. Declared in `<stdlib.h>`.
 - **new:** new is an operator. No separate header file needed.

- **Initialization:**

- **malloc():** Memory allocation only. Doesn't call constructors for objects.
 - **new:** Memory allocation and initialization. Calls constructors for objects.

- **Usage with Arrays:**

- **malloc():** No special handling for arrays.
 - **new:** Supports array allocation. `new[]` is used for arrays, which can later be deallocated with `delete[]`.

- **Return Type:**

- **malloc():** Returns a `void*` pointer. Need explicit type-casting.
 - **new:** Returns a pointer to the type of object being allocated. No type-casting required.

- **Overloading:**

- **new:** Supports overloading to customize memory allocation behavior.
 - **malloc():** malloc() is not meant to be overloaded.

- **Usage in C++:**

- **malloc():** Can be used in C++ but not recommended due to lack of support for constructors.
 - **new:** Preferred in C++ for dynamic memory allocation because it supports constructors.

- **free() vs. delete:**

- **Type:**

- **free():** free() is a function. Declared in `<stdlib.h>`.
 - **delete:** delete is an operator. No separate header file needed.

- **Type of Memory:**

- **free():** Used to deallocate memory allocated with `malloc()` or `calloc()`.
 - **delete:** Used to deallocate memory allocated with `new`.

- **De-Initialization:**

- `free()`: Only deallocates memory. Doesn't call destructors for objects.
- `delete`: Deallocates memory and calls destructors for objects.
- **Usage with Arrays:**
 - `free()`: No special handling for arrays.
 - `delete`: Used with `delete[]` to deallocate memory allocated for arrays.
- **Overloading:**
 - `delete`: Supports overloading to customize memory deallocation behavior.
 - `free()`: `free()` is not meant to be overloaded.
- **Usage in C++:**
 - `free()`: Not used in C++. Deallocating memory allocated with `malloc()` or `calloc()` using `free()` in C++ can lead to undefined behavior if the object has non-trivial constructors or destructors.
 - `delete`: Preferred in C++ for deallocating memory allocated with `new` because it properly calls destructors for objects.

2. Write a code to allocate and deallocate memory for multi-dimensional array using new and delete?

```
```CPP
#include <iostream>

int** allocateArray(int rows, int cols) {
 int** arr = new int*[rows]; // Allocate memory for array of row pointers
 for (int i = 0; i < rows; ++i) {
 arr[i] = new int[cols]; // Allocate memory for each row
 }
 return arr;
}

void deallocateArray(int** arr, int rows) {
 for (int i = 0; i < rows; ++i) {
 delete[] arr[i]; // Deallocate memory for each row
 }
 delete[] arr; // Deallocate memory for array of row pointers
}
```

```
int main() {
 int rows = 3;
 int cols = 4;
 int** arr = allocateArray(rows, cols);

 for (int i = 0; i < rows; ++i) {
 for (int j = 0; j < cols; ++j) {
 arr[i][j] = i * cols + j; // Example initialization
 }
 }

 std::cout << "Array elements:" << std::endl;
 for (int i = 0; i < rows; ++i) {
 for (int j = 0; j < cols; ++j) {
 std::cout << arr[i][j] << " ";
 }
 std::cout << std::endl;
 }

 deallocateArray(arr, rows);
 return 0;
}
...
```

3. What is "this" pointer? Is it available for static, virtual, const and friend functions?

- \* The "this" pointer in C++ is a pointer that holds the address of the current object. It is a keyword that can be used within the member functions of a class to refer to the object on which the member function is called.
- \* The "this" pointer is implicitly available within the scope of non-static member functions of a class.
- \* It is commonly used to access member variables and member functions of the current object. For example:  
```cpp

```

class Example {
private:
    int x;
public:
    void setX(int x) {
        this->x = x; // "this" pointer is used to differentiate between member variable and function parameter
    }
};
...
```cpp
int main() {
 Example ex;
 ex.setX(3);
}
...

```

\* **Static Member Functions:** "this" pointer is not available within static member functions because static member functions do not operate on specific class instances/objects.

\* **Virtual Member Functions:** "this" pointer is available in virtual member functions. When a virtual function is called through a pointer or reference to a base class, the "this" pointer points to the actual object type.

\* **Const Member Functions:** "this" pointer is available in const member functions. However, when a member function is declared as const, the type of "this" pointer is considered as a pointer to a constant object, so you cannot modify member variables inside const member functions.

\* **Friend Functions:** "this" pointer is not available within global friend functions because friend functions are not member functions of the class, even though they have access to the class's private and protected members. However, if friend function is member function of another class, it will have "this" pointer that will point to object of that class.

#### 4. Why we can not declare static member function constant or virtual?

\* In C++, static member functions cannot be declared as const or virtual.

\* **Const Member Functions:**

- Const member functions are functions that intended not to modify the state of the object on which they are called. They are declared using the `const` keyword at the end of the function declaration.

- Static member functions do not operate on specific class instances. They are not associated with any particular object, so it doesn't make sense to declare them as const because there's no object state to be modified or not modified.

\* \*\*Virtual Member Functions:\*\*

- Virtual member functions are used in polymorphism to achieve dynamic binding, allowing a derived class's implementation to be called through a base class pointer or reference.
- Static member functions do not participate in polymorphism because they are resolved at compile time based on the class name, not at runtime based on the object's type.
- Additionally, virtual functions are associated with specific objects and their behavior can vary depending on the object's type, which is not applicable to static member functions.

5. What is the need to write user defined destructor? When it should be declared as "virtual"?

- \* If base class pointer points, to dynamically allocated derived class object and derived class destructor is releasing any resource/memory, then base class destructor should be declared as virtual.
- \* The virtual destructor will ensure that first derived class destructor is called, and then the base class destructor is called. Finally, it will release the memory of dynamically allocated derived class object too. This will release all resources from base as well as derived classes.
- \* If destructor is not declared virtual, it will call base class destructor and release the memory of dynamically allocated derived class object. However, derived class destructor is not called; so resource/memory deallocated by derived class destructor will leak.

```
```cpp
#include <iostream>
class Base {
    int *bptr;
public:
    Base() {
        bptr = new int[3];
        std::cout << "Base constructor called" << std::endl;
    }
    virtual ~Base() {
        delete[] bptr;
    }
}
```

```
        std::cout << "Base destructor called" << std::endl;
    }
};
class Derived : public Base {
    int *dptr;
public:
    Derived() {
        dptr = new int[3];
        std::cout << "Derived constructor called" << std::endl;
    }
    ~Derived() {
        delete[] dptr;
        std::cout << "Derived destructor called" << std::endl;
    }
};

int main() {
    Base* ptr = new Derived(); // Using base class pointer to point to derived class object
    delete ptr; // Deleting object through base class pointer
    return 0;
}
...
```

6. Why constructor cannot be declared as virtual?

- * In C++, constructors cannot be declared as virtual because the virtual mechanism relies on the existence of a valid object.
- * When an object is being constructed, its virtual table (vtable) is not yet fully constructed or initialized. Consequently, the mechanism required to resolve virtual functions is not available during the construction process.
- * Also, virtual functions are designed to be called depending on type of object via pointer/reference. However, constructor is never called on pointer. It is called when object of the class is created (class name is given in code).

7. Explain dynamic_cast operator. When it is required? Explain with example.

* The `dynamic_cast` operator in C++ is used for performing safe downcasting (casting from a base class pointer or reference to a derived class pointer or reference). It allows you to check whether the object being cast is of a specific type at runtime, and if so, perform the cast safely.

* The `dynamic_cast` operator can only be used with pointers or references to polymorphic classes. A polymorphic class is a class that has at least one virtual function declared within it.

* Syntax:

```
```cpp
dynamic_cast<DerivedType*>(BaseType*);
```
```

* Example:

```
```cpp
#include <iostream>
class Base {
 int b;
public:
 virtual void fun1() { }
 virtual void fun2() { }
 virtual ~Base() {}
};
class Derived : public Base {
 int d;
public:
 virtual void fun1() { }
 void derivedMethod() {
 std::cout << "Derived method called" << std::endl;
 }
};
int main() {
 Base* basePtr = new Derived; // Upcasting: Derived object assigned to Base pointer
 //basePtr->derivedMethod(); // compiler error: due to Object slicing
}
```

```

// Attempting to downcast using dynamic_cast
Derived* derivedPtr = dynamic_cast<Derived*>(basePtr);
// Check if downcast was successful
if (derivedPtr != NULL) {
 // Downcast successful, call derived class method
 derivedPtr->derivedMethod();
} else {
 // Downcast failed, handle accordingly
 std::cout << "Dynamic cast failed" << std::endl;
}

delete basePtr; // Clean up allocated memory

return 0;
}
...

```

\* If `dynamic_cast` fails for the pointer, it returns `NULL`; However, if `dynamic_cast` fails for the references, it throws `bad_cast` exception.

## 8. How virtual function affects on size of object? How it is affected in single and multiple inheritance?

\* The presence of virtual functions in a class can affect the size of the object, primarily due to the addition of a virtual function table pointer (``vptr``).

\* **Single Inheritance:**

\* In single inheritance, where a class has only one direct base class, the addition of virtual functions typically increases the size of the object by the size of a pointer (usually 4 or 8 bytes depending on the architecture) to the virtual function table (``vtable``).

\* Example:

```

...cpp
class Base {
public:
 virtual void foo() {}
}

```

```
};
class Derived : public Base {
public:
 virtual void bar() {}
};
int main() {
 std::cout << "Size of Base: " << sizeof(Base) << std::endl; // Output: Size of Base: 8
 std::cout << "Size of Derived: " << sizeof(Derived) << std::endl; // Output: Size of Derived: 8 i.e. vptr of
Base is inherited to the Derived.
 return 0;
}
...
```

\* \*\*Multiple Inheritance:\*\*

\* In multiple inheritance, where a class has multiple direct base classes, each base class will have its own virtual function table pointer (`vptr`) and all these vptr will be inherited to the Derived class.

\* Example:

```
```cpp
```

```
class Base1 {
public:
```

```
    virtual void foo() {}
```

```
};
```

```
class Base2 {
```

```
public:
```

```
    virtual void bar() {}
```

```
};
```

```
class Derived : public Base1, public Base2 {
```

```
public:
```

```
    virtual void baz() {}
```

```
};
```

```
int main() {
```

```
    std::cout << "Size of Base1: " << sizeof(Base1) << std::endl;    // Output: Size of Base1: 8
```

```
std::cout << "Size of Base2: " << sizeof(Base2) << std::endl;    // Output: Size of Base2: 8
std::cout << "Size of Derived: " << sizeof(Derived) << std::endl; // Output: Size of Derived: 16
return 0;
}
...
```

9. What is the need to overload index operator? Explain with example?

* Overloading the index operator (`[]`) in C++ allows objects of a class to be treated like arrays, providing a way to access elements of the object using array syntax. This can make the code more intuitive and easier to read, especially when dealing with custom data structures or classes that represent collections of elements.

* Example:

```
```cpp
#include <iostream>

class MyArray {
private:
 int data[5]; // Internal array to store data
public:
 // Overloading the index operator
 int& operator[](int index) {
 // can check array bounds (index) here
 return data[index];
 }
};

int main() {
 MyArray arr;
 arr[0] = 10;
 arr[1] = 20;
 arr[2] = 30;
 std::cout << "arr[0]: " << arr[0] << std::endl;
}
```

```

 std::cout << "arr[1]: " << arr[1] << std::endl;
 std::cout << "arr[2]: " << arr[2] << std::endl;
 return 0;
 }
 ...

* Since operator return type is by reference, it can be used on left-side or right-side of the assignment operator.
    ```cpp
    y = arr[i];
        // y = arr.operator[](i);
    arr[i] = x;
        // arr.operator[](i) = x;
    ...

```

10. What is the difference between pointer and reference?

Pointers and references in C++ are both used to indirectly access objects, but they have some key differences:

- * Syntax:**
 - Pointer:** Pointers are declared using the `*` symbol. They store the memory address of another object. `cpp int x = 10; int* ptr = &x;`
 - Reference:** References are declared using the `&` symbol. They are aliases for other objects and must be initialized when declared. `cpp int x = 10; int& ref = x;`
- * Null Assignment:**
 - Pointer:** Pointers can be assigned the value `NULL` to indicate that they are not pointing to any valid memory location.
 - Reference:** References cannot be `NULL`. They must be initialized to refer to a valid object upon declaration.
- * Reassignment:**
 - Pointer:** Pointers can be reassigned to point to different objects (if not const pointer).
 - Reference:** References cannot be reassigned to refer to a different object after initialization. They are fixed aliases.
- * Dereferencing:**
 - Pointer:** Pointers are dereferenced using the `*` operator to access the value stored at the memory address they point to. `cpp int value = *ptr;`
 - Reference:** References are automatically dereferenced when used. They behave syntactically like the object they refer to. `cpp int value = ref; // Cannot explicitly dereference -- Auto dereferenced`
- * Memory Management:**
 - Pointer:** Pointers require manual memory management, such as allocation and deallocation using `new` and `delete`.
 - Reference:** References do not involve memory management. They are just aliases for existing objects.
- * Arithmetic Operations:**
 - Pointer:** Pointers support arithmetic operations such as incrementing and decrementing to move to different memory locations.
 - Reference:** References do not support arithmetic operations. They are fixed aliases and cannot be manipulated like pointers.
- * Function Parameters:**
 - Pointer:** Pointers are commonly used to pass parameters to functions by address, allowing the function to modify the original object.
 - Reference:** References can also be used to pass parameters by reference. They provide a cleaner syntax compared to pointers.

** However, internally references are pointer (that are auto-dereferenced) and size of reference is always 8 bytes.*

```

    ```cpp char z = 'A'; // global var

```

```

class Test {
 char &x;
public:
 Test() : x(z) { ... }
};

int main() {
 cout << sizeof(Test) << endl; // 8 bytes
 return 0;
}
...

```

## 11. What is diamond problem? How to solve it?

\* The diamond problem is a common issue that arises in multiple inheritance when a class inherits from two or more classes that have a common ancestor. This situation creates ambiguity in the inheritance hierarchy, as there can be multiple paths to access the same member from the common ancestor.

\* Consider the following inheritance hierarchy:

```

...
 A
 / \
B C
 \ /
 D
...

```

\* If class `A` has a member function `foo()`, then it is inherited to class `D` from two paths i.e. class `B` and `C`.

```

...cpp
class A {
public:
 void foo() { std::cout << "A::foo()" << std::endl; }
};

```

```
class B : public A {
public:
};

class C : public A {
public:
};

class D : public B, public C {
};

int main() {
 D d;
 d.foo(); // Call is ambiguous
 return 0;
}
...

```

\* Calling foo() on D class object leads to ambiguity error.

\* There are two ways to solve the diamond problem in C++:

1. **Virtual Inheritance:**

- Virtual inheritance is a feature in C++ that allows a class to inherit from a common base class only once, regardless of how many times it appears in the inheritance hierarchy.

- By using virtual inheritance, ambiguity is avoided, and there is only one instance of the common base class shared among all derived classes.

\* Example:

```
```cpp
```

```
class A {
public:
    void foo() { std::cout << "A::foo()" << std::endl; }
};
```

```
class B : virtual public A {
```

```
};

class C : virtual public A {
};

class D : public B, public C {
};

int main() {
    D d;
    d.foo(); // Output: A::foo()
    return 0;
}
...
```

2. **Scope Resolution Operator:**

- You can use the scope resolution operator (`::`) to explicitly specify which version of the member function should be called.

- This approach is less preferred as it does not resolve the underlying ambiguity and may lead to code maintenance issues.

- Example:

```
```cpp
int main() {
 D d;
 d.B::foo(); // Output: A::foo()
 return 0;
}
...
```

12. What is shallow copy and deep copy? How it is implemented in C++? Explain with example.

\* Shallow copy and deep copy are two ways of copying objects in C++.

#### 1. \*\*Shallow Copy:\*\*

- Shallow copy creates a new object and copies all the data members of the original object to the new object.
- If the data members of the original object include pointers, shallow copy only copies the addresses of the pointers, not the data pointed to.
- As a result, both the original object and the copied object share the same dynamically allocated memory.
- Changes made to the data pointed to by one object affect the other object.

#### 2. \*\*Deep Copy:\*\*

- Deep copy creates a new object and copies all the data members of the original object to the new object.
- If the data members of the original object include pointers, deep copy allocates new memory for the pointers and copies the data pointed to by the pointers to the new memory locations.
- As a result, each object has its own dynamically allocated memory, and changes made to one object do not affect the other object.

\* In C++, shallow copy and deep copy can be implemented manually by overloading the copy constructor and the assignment operator. The default copy constructor and assignment operator does shallow copy,

\* Here's an example demonstrating shallow copy and deep copy:

```
```cpp
#include <iostream>
class DynamicArray {
private:
    int* data;
    int size;
public:
    DynamicArray(int sz) {
        size = sz;
        data = new int[size];
        for (int i = 0; i < size; ++i)
            data[i] = i + 1;
    }
}
```

```
DynamicArray(const DynamicArray& other) {
    size = other.size;
    data = new int[size];
    for (int i = 0; i < size; ++i)
        data[i] = other.data[i];
}

DynamicArray& operator=(const DynamicArray& other) {
    if (this != &other) {
        delete[] data;
        size = other.size;
        data = new int[size];
        for (int i = 0; i < size; ++i)
            data[i] = other.data[i];
    }
    return *this;
}

~DynamicArray() {
    delete[] data;
}

void display() {
    for (int i = 0; i < size; ++i) {
        std::cout << data[i] << " ";
    }
    std::cout << std::endl;
}

};

int main() {
    DynamicArray arr1(5);
    DynamicArray arr2 = arr1; // copy constructor
    DynamicArray arr3(arr1); // copy constructor
}
```

```
DynamicArray arr4(4);
arr3 = arr1; // assignment operator

std::cout << "Original array: ";
arr1.display();
std::cout << "Copied array: ";
arr2.display();
std::cout << "Assigned array: ";
arr4.display();
return 0;
}
...
```

13. What is conversion function? Which are conversion functions in C++?

- * A conversion function in C++ is a member function of a class that is used to convert an object of that class to another type.
- * It allows objects of a user-defined type to be implicitly or explicitly converted to another type.
- * There are two types of conversion functions in C++ i.e. conversion operator and single argument constructor.

```
```cpp
class Rational {
public:
 Rational(int num, int den) {
 numerator = num;
 denominator = den;
 }
 Rational(int num) {
 numerator = num;
 denominator = 1;
 }
 operator double() const {
 return (double)numerator / denominator;
 }
};
```

```
 }
private:
 int numerator;
 int denominator;
};

int main() {
 Rational r(3, 2);
 double d = r; // Implicit conversion to double
 // d = r.operator double();
 int x = 5;
 Rational s;
 s = x; // Implicit conversion from int to Rational
 // x --> anonymous Rational object
 // s = anonymous object (assignment)
 // anonymous object destroyed
 return 0;
}
...

```

\* Note that, both conversion functions are called automatically (in above example). To avoid this implicit conversion use `explicit` keyword before these conversion functions. Now they will be invoked only when casting is done.

```
```cpp
int main() {
    Rational r(3, 2);
    double d = static_cast<double>(r); // Explicit conversion to double
    // d = r.operator double();
    int x = 5;
    Rational s;
    s = static_cast<Rational>(x); // Explicit conversion from int to Rational
    // x --> anonymous Rational object
    // s = anonymous object (assignment)
    // anonymous object destroyed
    return 0;
}

```

```
}  
...
```

14. What is smart pointer? Which are smart pointers in C++?

- * Smart pointers are objects that behave like pointers but provide automatic memory management.
- * They help prevent memory leaks and manage the lifetime of dynamically allocated objects.
- * Smart pointers are part of the C++ Standard Library and are implemented as template classes.
- * The main smart pointers in C++ are:

1. `**std::unique_ptr**`

- ``std::unique_ptr`` is a smart pointer that owns a dynamically allocated object and ensures that the object is deleted when the ``unique_ptr`` goes out of scope or is reset.
- It is unique in that it cannot be copied or shared. It can only be moved.
- It is lightweight and efficient because it does not incur the overhead of reference counting.

```
```cpp  
#include <memory>
#include <iostream>

int main() {
 std::unique_ptr<int> ptr(new int(42));
 std::cout << *ptr << std::endl;
 return 0; // ptr automatically deletes the allocated memory when it goes out of scope
}
...
```

##### 2. `**std::shared_ptr**`

- ``std::shared_ptr`` is a smart pointer that manages the ownership of a dynamically allocated object using reference counting.
- It allows multiple ``shared_ptr`` objects to share ownership of the same dynamically allocated object.
- The object is deleted when the last ``shared_ptr`` pointing to it is destroyed or reset.

```
```cpp
```

```
#include <memory>
#include <iostream>

int main() {
    std::shared_ptr<int> ptr1(new int(42));
    std::shared_ptr<int> ptr2 = ptr1;
    std::cout << *ptr1 << " " << *ptr2 << std::endl;
    return 0; // Both ptr1 and ptr2 share ownership of the allocated memory
}
...
```

3. `std::weak_ptr`

- `std::weak_ptr` is a smart pointer that provides a non-owning reference to an object managed by `std::shared_ptr`.
- It does not participate in reference counting and does not keep the object alive.
- It is used to break cyclic dependencies between `std::shared_ptr` objects.

```
```cpp
#include <memory>
#include <iostream>

int main() {
 std::shared_ptr<int> ptr = std::make_shared<int>(42);
 std::weak_ptr<int> weakPtr = ptr;
 if (auto sharedPtr = weakPtr.lock()) {
 std::cout << *sharedPtr << std::endl; // Access the object if it still exists
 }
 return 0; // weakPtr does not affect the lifetime of the allocated memory
}
...
```

15. What is STL? Explain different components in STL with examples?

\* STL stands for Standard Template Library. It is a powerful set of C++ template classes and functions that provide reusable, efficient, and flexible algorithms and data structures. The STL is part of the C++ Standard Library and is widely used for various programming tasks, such as data manipulation, sorting, searching, and more.

\* The main components of the STL include:

- \* Containers
- \* Iterators
- \* Algorithms

\* **Containers:**

\* Containers are data structures that store and manage collections of objects. There are several types of containers in the STL, each with different characteristics and use cases. Some common containers include:

- **vector:** Dynamic array that can resize itself automatically.
- **list:** Doubly linked list that allows for efficient insertion and deletion of elements.
- **deque:** Double-ended queue that supports insertion and deletion at both ends.
- **stack:** LIFO (Last In, First Out) data structure.
- **queue:** FIFO (First In, First Out) data structure.
- **set:** Collection of unique, sorted elements.
- **map:** Collection of key-value pairs, where each key is unique.

\* Example:

```
```cpp
#include <iostream>
#include <vector>
#include <set>

int main() {
    // Vector example
    std::vector<int> vec = {1, 2, 3, 4, 5};
    vec.push_back(6);
    std::cout << "Vector size: " << vec.size() << std::endl;

    // Set example
```

```
std::set<int> mySet = {3, 1, 4, 1, 5, 9, 2, 6, 5};
std::cout << "Set size: " << mySet.size() << std::endl;

return 0;
}
...
```

2. **Iterators:**

Iterators are objects that provide a way to iterate over the elements of a container in a sequential manner. They act as pointers to elements within the container and support various operations like dereferencing, incrementing, and decrementing.

Example:

```
```cpp
#include <iostream>
#include <vector>

int main() {
 std::vector<int> vec = {1, 2, 3, 4, 5};

 // Using iterators to iterate over the vector
 for (auto it = vec.begin(); it != vec.end(); ++it) {
 std::cout << *it << " ";
 }
 std::cout << std::endl;

 return 0;
}
...
```

## 3. **Algorithms:**

Algorithms are functions that operate on containers and perform various operations such as sorting, searching, transforming, and modifying elements. STL provides a wide range of algorithms that can be used with different types of containers.

Example:

```
```cpp
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> vec = {5, 2, 7, 1, 9, 3};

    // Sorting the vector using std::sort algorithm
    std::sort(vec.begin(), vec.end());

    // Displaying the sorted vector
    for (auto elem : vec) {
        std::cout << elem << " ";
    }
    std::cout << std::endl;

    return 0;
}
```
```

These are some of the main components of the STL. By leveraging the containers, iterators, and algorithms provided by the STL, C++ programmers can write efficient and expressive code for a wide range of tasks.

16. What is difference between set, vector and map? How to traverse them? Explain with code.

The main differences between `set`, `vector`, and `map` in C++ STL lie in their characteristics, usage, and underlying data structures:

1. **Set:**

- A set is an associative container that stores unique elements in sorted order.
- It does not allow duplicate elements.

- Implemented using a self-balancing binary search tree (usually Red-Black Tree).
- Provides efficient insertion, deletion, and search operations.

#### \* \*\*Traversing a Set:\*\*

- Sets are typically traversed using iterators.
- Since sets are sorted, elements are traversed in sorted order.

```
```cpp
#include <iostream>
#include <set>

int main() {
    std::set<int> mySet = {3, 1, 4, 1, 5};

    // Traversing set using iterators
    for (auto it = mySet.begin(); it != mySet.end(); ++it) {
        std::cout << *it << " ";
    }
    std::cout << std::endl;

    return 0;
}
```
```

#### 2. \*\*Vector:\*\*

- A vector is a dynamic array that can resize itself automatically.
- Elements are stored in contiguous memory locations, allowing efficient random access.
- Allows duplicate elements.
- Provides efficient insertion and deletion at the end of the vector, but less efficient for insertion and deletion in the middle.

#### \* \*\*Traversing a Vector:\*\*

- Vectors can be traversed using iterators or index-based loops.
- Iterators provide a flexible way to traverse vectors.

```
```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> vec = {1, 2, 3, 4, 5};

    // Traversing vector using iterators
    for (auto it = vec.begin(); it != vec.end(); ++it) {
        std::cout << *it << " ";
    }
    std::cout << std::endl;

    // Traversing vector using index-based loop
    for (size_t i = 0; i < vec.size(); ++i) {
        std::cout << vec[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}
```
```

### 3. \*\*Map:\*\*

- A map is an associative container that stores key-value pairs.
- Each key is unique, and the elements are sorted based on the keys.
- Implemented using a self-balancing binary search tree or hash table.
- Provides efficient insertion, deletion, and search operations based on keys.

#### \* \*\*Traversing a Map:\*\*

- Maps are typically traversed using iterators, which provide access to both keys and values.

```
```cpp
#include <iostream>
```

```
#include <map>

int main() {
    std::map<int, std::string> myMap = {{1, "one"}, {2, "two"}, {3, "three"}}; // C++17

    // Traversing map using iterators
    for (auto it = myMap.begin(); it != myMap.end(); ++it) {
        std::cout << it->first << ": " << it->second << std::endl;
    }

    return 0;
}
```

Adadvanced Java

Spring vs Spring Boot

- Spring Boot = Spring framework + Embedded Web Server + Auto Configuration - XML Configuration - Jar Conflicts
- Spring Boot is not a standalone framework. It is combination of various frameworks in spring platform like spring-core, spring-webmvc, spring-data, etc.

Count Spring Beans in the application.

- Spring Boot actuators
 - Predefined REST endpoints used to monitor/configure the applications.
 - /beans returns all available beans in our BeanFactory. Unlike /auditevents, it doesn't support filtering.
 - <https://localhost:8080/beans>
- BeanPostProcessor
 - Custom bean postprocessor.
 - Called when each new bean is created -- part of bean life cycle.

```
@Component
public class BeanCounterPostProcessor implements BeanPostProcessor {
    public static int beanCount = 0;
    public Object postProcessBeforeInitialization(Object bean, String beanName) throws BeansException {
        return bean;
    }
    public Object postProcessAfterInitialization(Object bean, String beanName) throws BeansException {
        beanCount++;
        return bean;
    }
}
```

How to implement CommandLineRunner in Spring Boot application?

```
```Java
@SpringBootApplication
public class MyApp implements CommandLineRunner {
 public static void main(String[] args) {
 SpringApplication.run(MyApp.class, args);
 }

 @Autowired
 private ApplicationContext ctx;
 @Autowired
 private BookDao bookDao;
 public void run(String[] args) {
 System.out.println("Hello, Boot!");
 // ... using dao
 // ... use ctx.getBean()
 }
}
```
```

Web Server vs Application Server

- Web Server = Web container + Additional services
 - Web container = Servlet engine + JSP engine
 - Additional services = Connection pool + JNDI + SSL + ...
 - Light -- Faster
 - e.g. Tomcat, ...
- Application Server = Web container + EJB container + Additional services
 - Web container = Servlet engine + JSP engine
 - Additional services = Connection pool + JNDI + SSL + ...
 - EJB container = Execution of EJB
 - EJB -- Scalable Business Logic components

- Heavy -- Need high resources
- e.g. JBoss 5, WebLogic, WebSphere, ...

How to use JNDI in Java web application?

- Java Naming and Directory Interface
- Like Java RMI
 - A java object is given some name and registered it into rmi registry.
 - The client get info from rmi registry and creates a proxy for the object (through interface).
 - The client calls methods of that remote java object (through proxy).
- JNDI is typically a service provided by web server -- to identify a wellknown object e.g connection pool.
- Tomcat -- Connection Pool -- appln's META-INF/context.xml

```
<Resource name="jdbc/myappdb"
  auth="Container"
  type="javax.sql.DataSource"
  username="root"
  password="password"
  driverClassName="com.mysql.cj.jdbc.Driver"
  url="jdbc:mysql://localhost:3306/mysql"/>
```

- In our application (servlet/jsp):

```
InitialContext ctx = new InitialContext();
DataSource dataSrc = (DataSource)ctx.lookup("java:/comp/env/jdbc/myappdb");
// use dataSrc
```

Explain Hibernate Relations/Associations.

- ManyToOne

- Many Emps have 1 Dept.
- Default Fetch: EAGER

```
@Entity
class Emp {
    // ...
    @ManyToOne
    @JoinColumn(name="deptno") // FK column in Emp table
    private Dept dept;
}
```

- OneToMany

- 1 Dept has many Emps.
- Default Fetch: LAZY

```
@Entity
class Dept {
    // ...
    @OneToMany(mappedBy="dept")
    private List<Emp> empList;
}
```

- ManyToMany

- 1 Emp have many Meetings.
- 1 Meeting have many Emps.
- Default Fetch: LAZY
- Tables: Emp (empno, ename, ...), Meeting (meetno, topic, ...), EmpMeeting(empid, meetid)

```
@Entity
class Emp {
```

```
// ...
@ManyToMany
@JoinTable(name="EmpMeeting",
    joinColumns={@JoinColumn(name="empid")},
    inverseJoinColumns={@JoinColumn(name="meetid")})
private List<Meeting> meetingList;
}

@Entity
class Meeting {
    // ...
    @ManyToMany(mappedBy="meetingList")
    private List<Emp> empList;
}
```

- OneToOne
 - 1 Emp have 1 permanent Address.
 - Default Fetch: EAGER

```
@Entity
class Emp {
    // ...
    @OneToOne
    @JoinColumn(name="empid") // FK column in Address table
    private Address addr;
}
```

How Garbage Collection in Java works?

- Internally use mark and sweep algorithm.
- GC runs in two phases.
 - Phase 1: Mark -- go through all objects and mark them to be deleted (if not referenced by any other object).

- Phase 2: Invoke finalize() (if present) and delete the object.
- GC check references of each object at well-known places (roots).
 - Local variables
 - Static Fields
 - Other object fields

How to make a custom Data Source in Spring Boot application?

- When your data source bean is defined, Spring boot will skip auto-config of default data source bean.

```
@Configuration
public class AppConfig {
    @Bean
    public DataSource dataSource() {
        DriverManagerDataSource ds = new DriverManagerDataSource();
        ds.setDriverClass("com.mysql.cj.jdbc.Driver");
        ds.setUsername("root");
        ds.setPassword("manager");
        ds.setUrl("mysql://localhost:3306/test");
        // ...
        return ds;
    }
}
```

Explain Spring Security concepts.

- <https://www.youtube.com/watch?v=sm-8qfMWEV8&list=PLqq-6Pq4ITTYTEooakHchTGglSvkZAJnE>

Why to use JWT in REST applications? How to use them?

- JSON Web Token
- Used with REST services (-- REST is stateless).
- To identify the user -- keep user identity at the client end -- so that client cannot modify/tamper it.

- Step 1: Authentication process
 - Client --> Credentials --> Server --> Verify details and generate JWT --> Send to client.
- Step 2: For each protected resource request (Spring -- using Filter)
 - Client (with token) --> Read the token, verify the token and attach client info to current request --> Resource

What is CORS error?

- Cross Origin Resource Sharing
- Refer slides

What is @Transactional? Why to add on @Service?

- Spring Data JPA --> Hibernate ORM
 - By default, autocommit=false --> need to commit explicitly.
- Transaction mgmt can be done manually in hibernate.

```
try {  
    Transaction tx = session.beginTransaction();  
    // ...  
    tx.commit();  
}  
catch(Exception ex) {  
    tx.rollback();  
}
```

- Spring provides **TransactionManager** -- automate transaction process using AOP -- @Transactional
 - AOP -- preprocessing -- start tx.
 - AOP -- postprocessing -- commit/rollback tx.
- @Transactional can be written at DAO or Service layer.
- Usually, one Business logic needs multiple daos.

```
@Service
public class OrderService {
    @Transactional
    public Order placeOrder1(int prodId, int userId) {
        prod = prodDao.findById(prodId); // tx
        user = userDao.findById(userId); // tx
        Order order = new Order();
        order.setProduct(prod);
        order.setUser(user);
        return orderDao.save(order);    // tx
    }

    public Order placeOrder2(int prodId, int userId) {
        prod = prodDao.findById(prodId); // tx 1
        user = userDao.findById(userId); // tx 2
        Order order = new Order();
        order.setProduct(prod);
        order.setUser(user);
        return orderDao.save(order);    // tx 3
    }
}
```

Why is @Transactional is not written in Spring boot application?

- DAO/Repository implementation -- pre-defined SimpleJpaRepository class -- @Transactional(readOnly=true)
 - Hence each method in DAO -- runs in individual tx -- by default

How to handle Lazy Initialization Exception?

- Method 1:
 - `spring.jpa.properties.hibernate.enable_lazy_load_no_trans=true`
- Method 2:
 - Fetch all data within the transaction itself -- by calling methods or using custom queries.

What is IoC and DI?

- The Inversion of Control is a fundamental principle used by frameworks to invert the responsibilities of flow control in an application.
- Dependency Injection is the pattern/mechanism used to provide dependencies to an application's class. Dependency objects are injected into dependent object (by the container) to initialize it, so that it will be ready to execute business logic.